V VAULTDROP

PROJECT SPEC
V1.0 · 2026

A SELF-REFERENCE DOCUMENT

Vaultdrop.

A private file vault for shared computers. Upload from your phone, download from the lab — nothing left behind.

Zero Cost | 24h Auto-Delete | QR Login | 20 MB Max | RLS Isolated

0	3	24h
MONTHLY COST	CORE SERVICES	FILE EXPIRY

NEXT.JS · SUPABASE · VERCEL

BRAND: #014BAA · #F8F3F0

CONTENTS

What's Inside

This document is the single source of truth for the Vaultdrop project. It captures the problem, the stack, the data model, every feature decision (with rationale), the color system, security analysis, cost breakdown, and a phased build plan. Keep it open while building — every section is designed to be scannable.

01	Project Overview	3
02	Tech Stack	4
03	How Privacy Actually Works	5
04	Database Schema & Storage Layout	6
05	Core Locked-In Features	7
06	Auto-Logout & Panic Logout	8
07	File Preview & Type Filter	9
08	Storage Meter & Bulk Actions	10
09	Icons, Expiry, Empty State, DnD	11
10	Color Palette & Design System	12
11	Security on Public PCs	13
12	Free Tier & Cost Analysis	14
13	Build Roadmap (Phased)	15
14	What We Deliberately Left Out	16
15	Getting Started Checklist	17

01 — OVERVIEW

The Problem & The Pitch

Shared computer labs force students to log into personal accounts just to access their own files — exposing them to keyloggers, cached sessions, shoulder surfing, and stale browser tabs that the next person can reopen.

Every college student has lived this scenario: you walk into the lab, need to print one PDF from your drive or send one photo to your phone, and the only path the institution gives you is to log into your Google Drive, Dropbox, or email on a public machine you don't control. You type your password on a keyboard that may be running a keylogger in the background. You leave a session token cached in a browser that the next student can reopen. You worry about

shoulder surfing while your inbox is visible. And when you finally finish, you're never quite sure whether you actually signed out everywhere — or just closed the tab.

Vaultdrop removes that risk entirely. It's a private, web-based file vault with three principles: *privacy-first* (every user's files are isolated at the database level, not just in application code), *zero-cost* (runs entirely on free tiers — no credit card, no surprise bills), and *frictionless* (QR login from your phone means you never type a password on a shared keyboard). Files auto-delete after 24 hours, so even if a session leaks, the blast radius is tiny. There is no sharing, no social layer, no public profile — the product is unapologetically single-user.

The use case is narrow on purpose: move a file from your phone to the lab PC, or from the lab PC to your phone, without ever typing credentials into the lab machine. That's it. Twenty megabytes per file is enough for documents, photos, code, PDFs — and blocks the video-abuse vector. Twenty-four hours is enough for a working day, and short enough that storage stays clean without manual maintenance.

02 — STACK

Tech Stack & Why Each Piece

The stack is deliberately small — three services total, all on free tiers, all chosen because they remove entire categories of work rather than add features. Every piece has a specific job; nothing is included "just in case."

Frontend

Next.js 16 with React handles routing, server-side rendering for the login page (faster first paint on slow lab Wi-Fi), and API routes for the QR pairing flow. **TypeScript** gives compile-time safety on file metadata shapes. **Tailwind CSS 4** with **shadcn/ui** ships accessible upload zones, dialogs, and toasts without writing custom CSS for every component. The entire front-end deploys as static + serverless functions — no always-on server to pay for.

Backend (BaaS)

Supabase is the only backend. It bundles three things that would otherwise be three separate services: a PostgreSQL database for file metadata, an Auth service for users (email/password + magic links + the QR pairing logic), and a Storage service (S3-compatible) for the actual file bytes. The single most important reason to pick Supabase over Firebase is **Row Level Security (RLS)** — policies enforced inside Postgres itself, meaning even a bug in your API code cannot leak another user's files. We will rely on this heavily.

Hosting

Vercel hosts the Next.js app on the free Hobby tier. Every `git push` triggers a deploy, you get a `vaultdrop.vercel.app` URL with automatic HTTPS, and a global CDN handles static assets. No DevOps work — the deployment pipeline is one command.

LAYER	CHOICE	FREE TIER COVERS	WHY THIS & NOT THE ALTERNATIVE
Frontend framework	Next.js 16 + React	Unlimited	API routes + SSR + file-based routing in one package
Styling	Tailwind 4 + shadcn/ui	Unlimited	Copy-paste components, no CSS-in-JS bundle weight
Database + Auth + Storage	Supabase	500 MB DB · 1 GB storage · 50K MAU	RLS at the DB layer; Firebase lacks this and locks you in
Hosting	Vercel Hobby	100 GB bandwidth · unlimited builds	Zero-config deploys, automatic HTTPS, edge CDN
Source control	GitHub (free)	Unlimited public repos	Triggers Vercel deploys on push

Alternatives We Rejected

- **Firebase** — generous free tier but no RLS equivalent (only security rules in code), Google can suspend accounts without warning, and Firestore's query model is restrictive for the metadata filtering we need.
- **Cloudflare Pages + Workers + R2 + D1** — actually a better free tier (10 GB R2 storage, no egress fees) but steeper learning curve. Worth revisiting if Supabase gets squeezed.
- **PocketBase self-hosted on Render** — single-binary delight, but free Render tier sleeps after 15 min of inactivity. Cold starts of 30+ seconds kill the "quick file grab" use case.
- **Plain Express + S3 + Auth0** — you'd spend the first week wiring auth, sessions, and storage. Not worth it for a solo project.

> **ARCHITECTURE IN ONE LINE**
>
> Browser → Vercel (Next.js + API routes) → Supabase (Postgres for metadata, Auth for sessions, Storage for bytes). All traffic HTTPS, all auth via HTTP-only cookies holding JWTs, all file access mediated by RLS.

03 — PRIVACY

How Privacy Actually Works

Privacy is not a feature flag — it's enforced at three layers, each of which would survive the failure of the others. This is called *defense in depth*, and it's the only honest way to make a privacy claim on a public computer.

Layer 1: Authentication

When a user logs in (via QR or email/password), Supabase Auth issues a signed JWT that lives in an **HTTP-only cookie**. HTTP-only means JavaScript on the page cannot read it — the browser attaches it automatically to every request to your domain. This blocks the most common XSS attack vector (stealing session tokens via injected script). The JWT has a server-side expiry; even if a user forgets to log out on the lab PC, the token stops working after the configured lifetime.

Layer 2: Database Row-Level Security

Every row in the `files` table carries an `owner_id` column. We enable RLS on this table and write a policy that says, in plain English: "a user may only SELECT, INSERT, UPDATE, or DELETE rows where `owner_id = auth.uid()` ". This policy runs **inside Postgres**, not in your Next.js code. Even if your API route has a bug that forgets to filter by user, the database itself refuses to return another user's rows — the query returns an empty result set. This is the single most important security decision in the project.

Layer 3: Storage Bucket Policies

Files in Supabase Storage are organized under a path like `user_{uuid}/filename.ext` . The bucket has a policy that mirrors the database RLS: a request may only read or write objects whose path starts with `user_` followed by the requesting user's own ID. Even if an attacker somehow obtains a direct storage URL for someone else's file, the signed URL expires and the bucket policy refuses the request. Files are never publicly listable.

> **WHY THIS MATTERS ON A PUBLIC LAB PC**
>
> Lab networks are often sniffable (shared Wi-Fi, no encryption between you and the access point). HTTPS protects traffic in transit. But the real risk is the *browser itself* — cached tokens, autocomplete passwords, history. By combining HTTP-only cookies (no JS access), server-side JWT expiry (time-bounded), and RLS (database-enforced isolation), we make it so that even a fully compromised browser session can only damage the user's *own* vault, and only for the next 24 hours of file lifetime.

04 — DATA

Database Schema & Storage Layout

Three tables, one storage bucket. The schema is intentionally flat — we don't need folders, versions, or relationships because files expire in 24 hours and there's no sharing.

Table: `profiles` (linked to `auth.users`)

Created automatically on signup via a trigger. Stores per-user configuration and is the join point for RLS.

COLUMN	TYPE	PURPOSE
`id`	uuid (PK)	Matches `auth.users.id`
`storage_cap_mb`	int	Per-user cap, default 50 (MB)
`created_at`	timestamptz	Account age

Table: `files` (the main metadata table)

One row per uploaded file. RLS-enforced: every query automatically filters to `owner_id = auth.uid()` .

COLUMN	TYPE	PURPOSE
`id`	uuid (PK)	Public-safe identifier used in URLs
`owner_id`	uuid (FK)	Who owns this file — RLS anchor
`filename`	text	Original filename for display
`storage_path`	text	Path in Supabase Storage: `user_{uuid}/{id}_{filename}`
`size_bytes`	bigint	For storage meter and 20 MB cap enforcement
`mime_type`	text	For preview routing and file-type icon
`created_at`	timestamptz	Upload timestamp
`expires_at`	timestamptz	`created_at + interval '24 hours'`

Table: `pairing_sessions` (for QR login)

Short-lived rows that mediate the QR pairing flow — modeled on WhatsApp Web. The lab PC visits `/login` and the server generates a one-time `qr_token`, displaying it as a QR on the monitor. The user's phone (already logged in from home) scans that QR with its camera and submits the token along with its own JWT — the server marks the pairing approved and issues a fresh JWT for the lab PC. The lab PC, which has been polling the pairing status, receives the new JWT and logs in. Nothing is ever typed on the lab PC — not a URL, not a code, not a password.

COLUMN	TYPE	PURPOSE
`id`	uuid (PK)	Session identifier
`qr_token`	text (unique)	One-time token generated by the server for the lab PC, encoded in the QR
`user_id`	uuid (FK, nullable)	NULL when created; filled in when the phone scans and approves (binds the lab PC session to the phone's identity)
`status`	enum	`pending` → `approved` / `expired` / `denied`
`created_at`	timestamptz	When the lab PC first loaded the QR page
`expires_at`	timestamptz	2 minutes from creation — the lab PC page auto-refreshes a new QR if not scanned in time

Storage Bucket Layout

A single private bucket named `vault`. Files are stored under the prefix pattern `user_{owner_uuid}/{file_uuid}_{original_filename}`. The `user_` prefix is what the bucket policy matches against `auth.uid()` to enforce per-user isolation. A nightly cron (Supabase's `pg_cron` extension) deletes expired rows from `files` and removes the corresponding storage objects.

01

Lab PC: visit

User opens `vaultdrop.vercel.app` on the lab PC and clicks "Login with QR". Server generates a one-time `qr_token` and displays it as a QR on the monitor.

→

02

Phone: scan

User opens Vaultdrop on their phone (already logged in from home), taps "Scan Lab PC" — phone camera opens and scans the QR displayed on the lab PC monitor.

→

03

Phone: approve

Phone app POSTs the scanned `qr_token` along with its own JWT to `/api/pair/approve`. Server validates the phone's identity, marks pairing `approved`, binds the `user_id`.

→

04

Lab PC: JWT

Lab PC has been polling `/api/pair/status?token=...`; on `approved` it receives a fresh JWT, sets the HTTP-only cookie, redirects to `/dashboard`. Nothing was typed on the lab PC.

05 — CORE

The Non-Negotiable Features

These four decisions are locked. They shape every other choice in the project — schema, UX, security, and the build sequence. Each one solves a specific problem we'd otherwise have to design around.

A 20 MB File Size Cap

Hard limit at the upload layer (both client-side and server-side via Supabase Storage bucket config). Why 20 MB: it comfortably covers PDFs, Word docs, Excel sheets, code files, and high-res photos, while blocking the obvious abuse vector of using the vault as free video storage. At 20 MB per file, the 1 GB Supabase free tier supports ~50 concurrent files — combined with 24h expiry, that's plenty of headroom for a personal vault.

B 24-Hour Auto-Delete

Every file row carries an `expires_at = created_at + interval '24 hours'`. A nightly Supabase `pg_cron` job deletes expired rows from the `files` table and removes the corresponding storage objects. This is the project's self-cleaning mechanism: no manual maintenance, no storage bloat, no stale files left behind on the lab PC. From a privacy angle, it also caps the blast radius of any session leak to 24 hours of file history at most.

C QR Code Login

The single most important UX decision in the project — modeled on WhatsApp Web. The lab PC never needs a camera, never asks you to type a URL-with-token, and never asks for a password. The flow: you walk up to the lab PC, open `vaultdrop.vercel.app` (bookmarked once, or just remember the short URL), and click "Login with QR". The page displays a QR on the monitor. You open Vaultdrop on your phone (already logged in from home, password typed in private), tap "Scan Lab PC", and point your phone's camera at the monitor. The phone app sends the scanned token plus its own JWT to the server; the server validates and issues a fresh JWT for the lab PC. The lab PC's polling picks up the approval, sets an HTTP-only cookie, and you're in — without ever typing a single character on the shared keyboard. This kills the keylogger threat entirely (nothing to log) and avoids the URL-typing fatigue that comes with token-in-URL schemes (typing `vaultdrop.vercel.app/login?token=abc123` is, as you noted, just as bad as typing a password). The QR token expires in 2 minutes and is one-time-use; if you don't scan in time, the page auto-refreshes a new QR.

D Brand Colors #014BAA + #F8F3F0

The deep navy `#014BAA` is the primary brand color — buttons, active states, headings, file-type accents. The warm cream `#F8F3F0` is the page background — softer than pure white, reduces eye strain in fluorescent-lit labs, and pairs visually with the cool blue via complementary temperature contrast. The full palette derived from these two is documented in Section 10.

06 — FEATURE

Auto-Logout with Extend Banner

On a public lab computer, "I forgot to log out" is the single most common security failure. Vaultdrop addresses this with two complementary mechanisms: a 10-minute inactivity timer that automatically ends the session, and a panic logout button for the cases where you need to disappear the screen instantly.

The Idle Timer

A client-side idle timer resets on `mousemove`, `keydown`, and `scroll` events. At the 9-minute mark (60 seconds before logout), a banner slides up from the bottom of the screen with a countdown: *"Logging out in 60s — [Extend]"*. Tapping Extend resets the timer to 10 minutes. If the user does nothing, at 0 the app forces a logout: clears the JWT cookie via a server call, wipes the cached file list from local state, and redirects to `/login`.

Defense in Depth

The client-side timer is the friendly layer — it gives the user a chance to extend. The hard layer is server-side: the JWT itself has a 30-minute absolute expiry from issuance, and the refresh token has a 15-minute sliding expiry. Even if the client timer is bypassed (e.g., user closes the laptop lid and reopens it 2 hours later), the refresh will fail and the app will redirect to login on the next API call. This combination of "soft client warning + hard server expiry" is the right pattern for shared devices.

Panic Logout Button

A small red floating button in the bottom-right corner, always visible. One tap triggers the same logout flow as the idle timer but with no confirmation dialog — the entire point is that it's instant. Use case: a professor walks behind you mid-upload. You tap the button, the browser is back on the login screen before they've finished walking past. The button has a subtle hover state but no label, to keep it visually unobtrusive until you need it.

WHY 10 MINUTES AND NOT 5 OR 30

5 minutes is too aggressive — users get logged out mid-task while reading a long PDF on their phone. 30 minutes is too generous — if you walk away from the lab PC without logging out, the next student has half an hour to snoop. 10 minutes is the sweet spot: long enough to read a document, short enough that an unattended terminal is locked before the next person sits down.

07 — FEATURE

File Preview & Type Filter

The number one reason students open their email on a lab PC is to double-check they actually attached the right file. Vaultdrop's preview solves that without requiring a download round-trip. Click a file row, a modal opens, you see the content immediately, you confirm, you download — or you close and pick another.

Supported Preview Types

TYPE	FORMATS	PREVIEW METHOD
Images	PNG, JPG, WebP, GIF	Direct `<img>` render, full-bleed in modal
PDF	.pdf	Inline `<iframe>` with the signed URL
Plain text	.txt, .md, .csv	Pre-formatted text block, monospace, scrollable
Code	.js, .ts, .py, .json, .html, .css	Syntax-highlighted via `shiki` or `prism`
Office docs	.docx, .xlsx, .pptx	No inline preview — show file icon + "Download to view" CTA
Archives	.zip, .rar, .7z	No preview — show file icon + size + item count if extractable

File Type Filter Chips

Above the file list, a horizontal row of filter chips: All Images PDF Docs Code Archive Other. Clicking a chip filters the list client-side instantly — no refetch, no spinner. The active chip is filled with the primary blue; inactive chips are outlined. This is small UX, but it's the difference between scrolling through 30 files looking for the one PDF and just tapping "PDF" and seeing it immediately.

WHY NO DOCX/XLSX INLINE PREVIEW

Rendering Office formats in the browser requires heavy libraries (docx-preview, sheetjs) that bloat the bundle. For a 24-hour-lifetime vault where the user almost always knows what they uploaded, the cost isn't justified. Showing a clear file-type icon and a download button is faster to build, faster to load, and visually consistent with the rest of the app.

08 — FEATURE

Storage Meter & Bulk Actions

Two features that make the vault sustainable at the free tier: a per-user storage meter that prevents any single user from monopolizing the 1 GB project quota, and bulk actions that make it painless to clear out expired-or-unwanted files in one motion.

Storage Usage Meter

A chip in the header shows `X.X MB / 50 MB used` with a thin horizontal progress bar underneath. The bar turns amber at 80% full and red at 95%. The meter updates live as uploads complete and as files are deleted — no page refresh required. The 50 MB per-user cap is enforced server-side: before an upload is accepted, the API checks `SUM(size_bytes) WHERE owner_id = auth.uid()` and rejects the upload if it would exceed the cap. This means even a malicious user can't bypass the limit by hitting the API directly.

50 MB	1 GB	~20
PER-USER CAP	PROJECT STORAGE	ACTIVE USERS SUPPORTED

Multi-Select Bulk Actions

Each file row has a checkbox on the left edge. A "Select all" toggle appears at the top of the list. When one or more files are selected, a bulk-action bar slides up from the bottom of the screen with two actions: **Download all** (zips the selected files server-side and returns a single archive) and **Delete all** (opens a confirm modal showing the count and total size, then deletes the rows and storage objects in a single transaction). On mobile, long-pressing a file enters multi-select mode — same pattern as iOS Files and Google Drive, so users already know the gesture.

Why Both Matter

Without the storage meter, one user uploading a few large PDFs could exhaust the 1 GB project quota and lock out everyone else. Without bulk actions, clearing out 15 expired files at the end of a study session would require 15 individual delete taps — enough friction that nobody would do it, leading to reliance on the 24h auto-delete as the only cleanup path. The meter makes the cap visible; bulk actions make compliance with it painless.

09 — FEATURE

Icons, Expiry, Empty State, Drag & Drop

Four small features that together make the difference between a vault that feels like a polished product and one that feels like a homework assignment. None of them is individually complex; all of them address a real friction point in the daily-use loop.

File Type Icons with Color Tags

Every file row shows a small colored icon on the left, keyed off the MIME type. PDF = red, Image = green, Document (docx/txt/md) = blue, Code = purple, Archive = amber, Other = grey. Visual scanning is dramatically faster than reading filenames, especially on a small phone screen in a hurry between classes. The colors are drawn from the extended palette (Section 10) and stay consistent across the file list, preview modal, and bulk-action bar.

"Expires In" Countdown

Each file row shows a small monospace timestamp like `expires in 23h 14m` under the filename. The countdown updates client-side every minute. In the last hour before expiry, the timestamp turns red — a visual cue that the file is about to disappear, so the user should download it now if they still need it. This eliminates the "wait, when did I upload this?" anxiety that comes with a 24h-lifetime vault, and it makes the auto-delete feel like a feature rather than a surprise.

Empty State with Clear CTA

When the vault has zero files (first visit, or after everything has expired), the dashboard shows a centered illustration of an open vault with the headline "Drag files here or tap to upload" and a subtle sub-line "Files auto-

delete after 24h". A large primary button below opens the file picker. The drag zone is full-width — the entire dashboard is the drop target, not just a small rectangle. Never show a blank table with column headers and no rows; that reads as "broken" rather than "empty".

Drag-and-Drop + Tap-to-Pick

Same upload component, two interaction modes. On desktop (lab PC), dragging a file over the browser window expands the drop zone to fill the screen with a subtle blue overlay and the text "Drop to upload". On mobile (phone), the same component shows a single large "Upload file" button that opens the native file picker — including the iOS/Android option to take a photo or pick from the camera roll. The component detects pointer type at runtime; no separate code paths to maintain.

> **THE INTERACTION PHILOSOPHY**
>
> These four features share a single goal: *reduce the number of taps between "I have a file" and "the file is in my vault"*. Color icons reduce visual scanning time. The expiry countdown reduces anxiety about file lifetime. The empty state reduces the cold-start friction. Drag-and-drop eliminates the file-picker round-trip on desktop. Each one saves a second or two; together they make the app feel instant.

10 — DESIGN

Color Palette & Design System

Two brand colors alone aren't enough for a real interface — you need neutrals, status colors, and tints that all harmonize. The full system below is derived from `#014BAA` (primary) and `#F8F3F0` (background), with neutrals kept warm to match the cream background rather than pure grey (which would clash).

`#014BAA` **Primary** Buttons, links, headings	`#013A85` **Primary Hover** 15% darker, button hover	`#E6EEF7` **Primary Light** Selected row, callouts
`#F8F3F0` **Background** Page background, warm cream	`#FFFFFF` **Surface** Cards, modals, upload zones	`#E5DCD4` **Border** Warm-tinted, 10% darker than bg
`#1A1A1A` **Text Primary** Body text, headings	`#6B6B6B` **Text Secondary** Captions, metadata	`#9B9B9B` **Text Muted** Timestamps, disabled
`#1B8738` **Success** Upload complete, signed in	`#C8102E` **Error** Delete, panic logout, failed	`#B8860B` **Warning** Expiring soon, extending

Typography

Two typefaces, no more. **Inter** for everything UI (body, headings, labels, buttons) at weights 400/500/600/700/800. **JetBrains Mono** for code, file IDs, expiry countdowns, and the storage meter — anywhere a tabular, monospace feel aids scanning. Type scale: 12 (caption) / 13.5 (body) / 15 (lead) / 16 (h3) / 22 (h2 small) / 26 (h2) / 88 (cover hero). Line-height 1.65 for body, 1.15 for display headings.

Spacing & Geometry

4-pixel base unit. Component radius: 8px for cards, 6px for buttons, 4px for inputs, 999px for pills. Shadows: a single subtle layer `0 2px 8px rgba(1,75,170,0.08)` on cards and modals — no layering, no glow, no neon. Borders are 1px in `#E5DCD4`. The overall feel is calm and quiet — the vault should look like a tool, not a marketing site.

Why This Palette Works

`#014BAA` is a cool deep blue. `#F8F3F0` is a warm cream. They sit on opposite sides of the color-temperature spectrum, which creates visual contrast without clashing — the blue pops against the warm background, and the cream softens the coldness of the blue. The neutrals all lean slightly warm (notice `#E5DCD4` is not pure grey) to maintain temperature consistency. Pure greys would feel out of place against the cream; pure white surfaces sit naturally because the cream is light enough to read as "off-white" itself.

— 11 — SECURITY

Security on Public PCs

Shared lab computers present a unique threat model that most web apps never have to consider. The attacker isn't a remote hacker — it's the next student who sits down at the same machine, or a piece of malware already installed on the lab image, or someone watching you type. Here's the threat list and how Vaultdrop mitigates each one.

THREAT	HOW VAULTDROP MITIGATES
Keyloggers on the lab machine capturing passwords or URL tokens	QR login (WhatsApp Web pattern) — the lab PC displays a QR on its monitor, the phone scans it with its camera. Nothing is ever typed on the lab PC: no URL-with-token, no password, no code. Even if a keylogger is running, there's nothing to capture.
Shoulder surfing while you type or while your files are visible	No password typed (QR flow). 10-minute auto-logout minimizes window of visibility. Panic logout button for instant screen-clear.
Cached sessions in the browser, reopened by next user via "Reopen closed tab"	JWT in HTTP-only cookie (not localStorage). Force-logout calls server to invalidate refresh token. Cache-Control: no-store on all API responses.
Browser history showing file URLs you visited	Files served via short-lived signed URLs (60s expiry). Even if URL is in history, it stops working almost immediately. Recommend users run the app in incognito/private window.
Network sniffing on shared lab Wi-Fi	HTTPS enforced by Vercel, HSTS header set. All traffic including file bytes encrypted in transit.
Stale tabs left open from previous user	JWT has 30-minute absolute expiry. Refresh fails silently. App redirects to login on next API call. No "you're still logged in" surprise.
Autocomplete & URL history on the lab browser remembering credentials or token URLs	Login form has `autocomplete="off"` . We also recommend incognito. QR login removes the password field entirely for the lab flow — the only thing ever typed on the lab PC is the bare URL `vaultdrop.vercel.app` (bookmarked once, never re-typed). No token URLs ever enter the address bar.
Database leak if Supabase is compromised	Files stored as binary blobs in Storage, not in the database. Metadata only. RLS still prevents cross-user access even with direct DB access.

Recommended User Behavior

Even with all the above mitigations, the strongest defense is user habit. Onboarding should remind users to: (1) use the lab browser's incognito/private mode, (2) always tap the panic logout button before walking away, (3) never use the "remember me" checkbox (we don't even offer one), and (4) if anything feels off — weird redirects, unexpected prompts — close the browser and report the machine to lab staff. The app can do a lot, but it can't fix a compromised lab image.

12 — COST

Free Tier & Cost Analysis

The whole project hinges on staying inside free tiers. Here's exactly what each service gives you, what happens at the limit, and the math on how many users that translates to.

SERVICE	FREE TIER LIMIT	WHAT HAPPENS WHEN EXCEEDED
Vercel Hobby bandwidth	100 GB / month	App pauses for the rest of the billing cycle, resumes on the 1st
Vercel Hobby builds	Unlimited (with queue)	Builds may wait in queue during peak hours
Supabase database	500 MB	Read-only mode — no new inserts until upgrade or cleanup
Supabase file storage	1 GB total	Uploads rejected with quota error
Supabase egress	2 GB / month	Requests throttled, may fail
Supabase auth users	50,000 MAU	Way beyond college-scale; never an issue
Supabase projects	2 free projects	Need to delete one to create a third
Supabase inactivity pause	7 days no API calls	Project pauses; first request after pause takes ~30 seconds to wake

The Math

Storage is the binding constraint. 1 GB project storage divided by 50 MB per-user cap equals 20 concurrent active users at full cap. In practice, most users will use far less than their cap (a few PDFs and photos), so realistic capacity is closer to 50–80 users. Combined with 24h auto-delete (which means files don't accumulate over time), this is plenty for a personal project, a small friend group, or a single college lab's worth of users.

Bandwidth is the secondary constraint. At an average 5 MB per file and 100 GB monthly bandwidth, that's roughly 20,000 file downloads per month — again, far beyond realistic usage for a small user base. The 2 GB Supabase egress is more constraining but only applies to file bytes (not metadata); divide by 5 MB average file size and you get 400 file downloads per month, which is the realistic ceiling if your entire user base is highly active.

Sustainability Checklist

- **Nightly cron** deletes expired files from `files` table and Storage — non-negotiable, prevents storage bloat.
- **Per-user 50 MB cap** enforced server-side — prevents any single user from monopolizing the project quota.
- **20 MB file size cap** — prevents a single upload from consuming 4% of total project storage.
- **Rate-limit uploads** — max 10 uploads per minute per user, blocks abuse patterns.
- **Keep-alive ping** — if your app has low traffic, set a cron-job.org hit to ping the API daily to prevent the 7-day inactivity pause.

> **WHEN TO CONSIDER PAYING**
>
> Upgrade triggers: (1) more than ~30 active users, (2) need a custom domain (~$12/year for a .com), (3) need files larger than 20 MB, (4) want to remove Vercel's 100 GB bandwidth cap. Supabase Pro is $25/month — wait until usage actually demands it. Don't pre-pay for capacity you don't yet need.

13 — ROADMAP

Build Roadmap (Phased)

Three phases, each independently shippable. The goal of each phase is to end with something you can actually use — not a half-built feature set. Solo-dev time estimates; adjust to your pace.

1

MVP · ~1 WEEK

Working upload + download + 24h expiry

Supabase project setup with RLS enabled. Email/password auth. Single-page dashboard: upload button, file list, download, delete. 24h expiry via `pg_cron` nightly job. Deploy to Vercel. **Test after Phase 1:** can you upload from your phone and download from your laptop? Yes → ship it.

2

CORE UX · ~1 WEEK

QR login + all the daily-use features

QR pairing flow (lab PC displays QR on monitor, phone scans with camera, phone approves via server, lab PC polls and receives JWT). Storage meter in header. File preview modal (images, PDF, code). Multi-select bulk actions. File type icons with color tags. Expiry countdown on each row. Empty state with CTA. Drag-and-drop on desktop. Sort + search + filter chips. **Test after Phase 2:** on the lab PC, do you reach the dashboard without typing anything other than the bare URL? Does the meter update live?

3

POLISH · ~3–5 DAYS

Auto-logout, panic button, toasts, PWA

10-minute idle timer with 60-second extend banner. Panic logout floating button. Active sessions indicator ("logged in on 2 devices"). Toast notifications for upload/delete/logout events. Optional: dark mode toggle. Optional: PWA manifest so the phone can "install" the vault as an app icon. **Test after Phase 3:** let the lab PC sit idle for 10 minutes — does it auto-logout? Tap panic — does it clear instantly?

What NOT to Build in Any Phase

If you find yourself wanting to add share links, folders, version history, collaborative editing, or email notifications — **stop**. Each one of these is a separate product. Vaultdrop's value comes from doing one thing well. The next section makes this explicit.

— 14 — SCOPE

What We Deliberately Left Out

Scope creep is the silent killer of personal projects. This section exists so that future-you, three weeks in and tired, doesn't second-guess the decisions made here. Every item below was considered and rejected for a specific reason.

FEATURE	WHY WE'RE NOT BUILDING IT
Share links	Defeats the privacy-first premise. Also adds a public attack surface (anyone with the link) that contradicts the entire value proposition.
Folders / labels	Files expire in 24 hours. Nobody is organizing a 24-hour-lifetime vault into folders. Adding folders adds complexity for zero user value.
Version history	Same reason. If a file is overwritten, the old version is irrelevant — both expire in 24h anyway.
Collaborative editing	This is a vault, not Google Docs. If you need to edit, download, edit, re-upload.
Comments / annotations	Single-user vault. Who would you be commenting with?
Public profiles	Privacy-first means no public surface. No usernames visible to other users, no avatars, no "people you may know".
Social features	Same as above. This is not a social product.
Email notifications	Sessions are 10 minutes long. By the time an email arrives, the user is already gone. Pure noise.
OCR / full-text search of file contents	Way out of scope. Filename search is enough for a 24-hour vault with at most ~50 files per user.
Video streaming / transcoding	20 MB cap blocks videos anyway. Not worth the engineering effort.
Mobile app (native)	The web app is installable as a PWA — that covers 95% of the "I want an app icon" need without maintaining a separate React Native codebase.
Admin panel	For a project with 20 users, SQL queries via Supabase Studio are sufficient. Don't build UI for admin work you'll do twice a month.

THE IRON RULE

If a feature does not directly serve the use case of "move a file between my phone and a shared computer safely", it does not belong in Vaultdrop. Period. When in doubt, leave it out — you can always add it later. You cannot easily un-add scope creep.

— 15 — START

Getting Started Checklist

Concrete next steps, in order. Tick them off as you go. Don't skip ahead — each phase depends on the previous one being complete.

ACCOUNT SETUP

- [ ] Create a GitHub account (if you don't have one) — free, required for Vercel and Supabase login
- [ ] Create a Vercel account at vercel.com — sign in with GitHub
- [ ] Create a Supabase account at supabase.com — sign in with GitHub
- [ ] Install Node.js 20+ (LTS) from nodejs.org
- [ ] Install Git on your dev machine

PROJECT SCAFFOLDING

- [ ] Run `npx create-next-app@latest vaultdrop` with TypeScript, Tailwind, App Router
- [ ] `cd vaultdrop` and install deps: `npm install @supabase/supabase-js @supabase/ssr`
- [ ] Initialize shadcn/ui: `npx shadcn@latest init`
- [ ] Create a new Supabase project from the dashboard — name it "vaultdrop"
- [ ] Copy the Project URL and anon key into `.env.local` as `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`

DATABASE & STORAGE SETUP

- [ ] In Supabase SQL Editor, run the schema script creating `profiles`, `files`, `pairing_sessions` tables
- [ ] Enable RLS on all three tables
- [ ] Write RLS policies: `USING (owner_id = auth.uid())` for `files`; same pattern for the others
- [ ] Create a Storage bucket named `vault` — set it to **private**, not public
- [ ] Write storage bucket policies mirroring the RLS rules (path prefix `user_${auth.uid()}/`)
- [ ] Enable `pg_cron` extension and schedule the nightly expiry cleanup job

BUILD PHASE 1 (MVP)

- [ ] Build the auth pages: `/login`, `/signup` (email + password for now)
- [ ] Build `/dashboard` with upload button, file list, download, delete
- [ ] Wire the upload to insert a row in `files` and upload bytes to Storage under `user_${user.id}/`
- [ ] Wire the file list to query `files` ordered by `created_at DESC`
- [ ] Wire delete to remove both the row and the storage object
- [ ] Run `npm run dev` and test the full loop locally

DEPLOY & TEST IN REAL CONDITIONS

- [ ] Push to GitHub: `git push origin main`
- [ ] Import the repo into Vercel — set the env vars in the Vercel dashboard
- [ ] Wait for the first deploy, visit your `vaultdrop.vercel.app` URL
- [ ] Test on actual lab PC: signup, upload, download, logout
- [ ] Test on phone (cellular, not Wi-Fi): same flow
- [ ] Wait 24 hours, verify the file is gone — confirms the cron is working

BUILD PHASE 2 & 3 (ITERATE)

- [ ] Add QR login flow (lab PC visits `/login` → server generates QR → phone scans with camera at `/scan` → phone POSTs token + JWT to `/api/pair/approve` → lab PC polls `/api/pair/status` and receives JWT)
- [ ] Add storage meter, file preview, multi-select, icons, expiry countdown, empty state, drag-and-drop
- [ ] Add 10-min auto-logout with extend banner, panic logout button, toasts
- [ ] Optional: add PWA manifest so phone can install the app
- [ ] Final test: full real-world flow — phone uploads, QR pair on lab PC, download, walk away, verify auto-logout fired

NOW GO BUILD IT.

Privacy isn't a feature. **It's the product.**

Every decision in this document — the 20 MB cap, the 24-hour expiry, the QR login, the RLS policies, the things we chose not to build — exists to make a single promise real: *your files are yours, on any computer, for as long as you need them, and not a second longer.*